

Name of the Student	K Ashok
Reg. No	192425322
GitHub Link	

SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL 2

Scenario-Based Questions

**COURSE NAME: Deep Learning COURSE CODE: MLA04 Faculty : Dr. Arul Raj
A.M Slot :**

COURSE OUTCOME COVERED

CO 2:Students will be able to apply supervised learning algorithms such as linear regression, classification models, decision trees, neural networks, support vector machines, and ensemble methods to solve real-world prediction and classification problems. They will gain the ability to select appropriate models, train them using datasets, and evaluate their performance. Students will also understand the strengths and limitations of different supervised learning approaches.(BTL-4)

CO Weightage : 15%

Assessment Tool Weightage: 40%

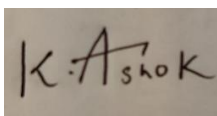
Duration: 90 minutes

Marks: 25

Code of Conduct:

I K Ashok (192425322) - (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student: K Ashok

Scenario-Based Questions CO-AT2

QUESTIONS: 05

Total Marks:25

1. Unsupervised Training of Neural Networks, Auto Encoders

A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

Collect ECG data: Gather thousands of ECG signals and preprocess them using normalization, noise removal, and fixed-length segmentation.

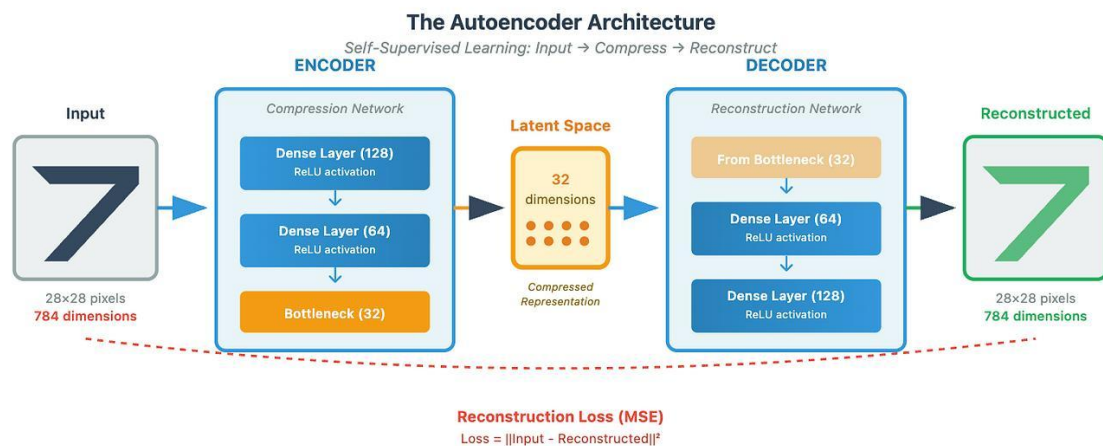
Train the autoencoder: ECG signals are given as both the input and reconstruction target. The **encoder** compresses each ECG into important features, while the **decoder** tries to reconstruct the original ECG.

Learn normal patterns: If training data is mostly normal, the network becomes good at reconstructing typical ECG patterns.

Calculate reconstruction error: Compare the original ECG with reconstructed ECG .

Detect anomalies: Normal ECG → low reconstruction error.
Unusual/abnormal ECG → typically higher reconstruction error.

Set an **anomaly threshold**. Records above the threshold are flagged for medical review.



Justification

Autoencoders are appropriate because **labeled ECG data is not required**, useful ECG features are learned automatically, and unusual patterns can be detected using reconstruction error.

Conclusion: An autoencoder-based anomaly detection system can identify potentially abnormal ECG records from large unlabeled datasets.

2. Auto Encoders, Representation Learning

A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.

Process

Step 1 – Input

Feed customer purchase data into the autoencoder.

Step 2 – Encoder

The encoder learns to compress the original high-dimensional data.

Here, is the compressed **latent representation**.

Step 3 – Bottleneck

The bottleneck contains fewer dimensions but tries to preserve the most useful patterns.

For example:

Step 4 – Decoder

The decoder reconstructs the original data from the compressed representation.

Training minimizes reconstruction error:

Step 5 – Remove decoder

After training, the decoder is no longer needed for segmentation.

Use:

Customer data → Encoder → Latent features

Step 6 – Customer segmentation

The compressed features can be passed to a clustering algorithm such as **K-Means**:

Possible segments could include high-value customers, occasional buyers, discount-oriented customers, etc.

Why use an autoencoder?

Unlike simple dimensionality-reduction techniques, neural autoencoders can learn **nonlinear relationships** in complex customer data.

Conclusion: The autoencoder performs representation learning and converts high-dimensional purchase data into compact features that can make customer clustering easier and more effective.

3. Width and Depth of Neural Networks, Activation Functions

A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions.

A shallow network may not learn sufficiently complex facial features.

For example:

Faces contain hierarchical features such as:

Edges → textures → eyes/nose/mouth → facial structures → identity

Recommended changes

1. Increase network depth

Instead of one or two hidden layers, use multiple hidden layers.

Example:

Earlier layers can learn simple features such as edges.

Middle layers learn features such as eyes, nose, and mouth.

Deeper layers learn high-level facial representations useful for recognizing individuals.

2. Increase width when necessary

Width refers to the **number of neurons/channels in a layer**.

For example:

Increasing width gives the network more capacity to learn different features, though excessive width can increase computational cost and overfitting.

3. Use nonlinear activation functions

Using only linear transformations prevents a deep network from learning complex nonlinear relationships.

A common choice is **ReLU**:

Example architecture:

ReLU is computationally efficient and generally works better in deep networks than sigmoid for hidden layers because it reduces problems associated with vanishing gradients.

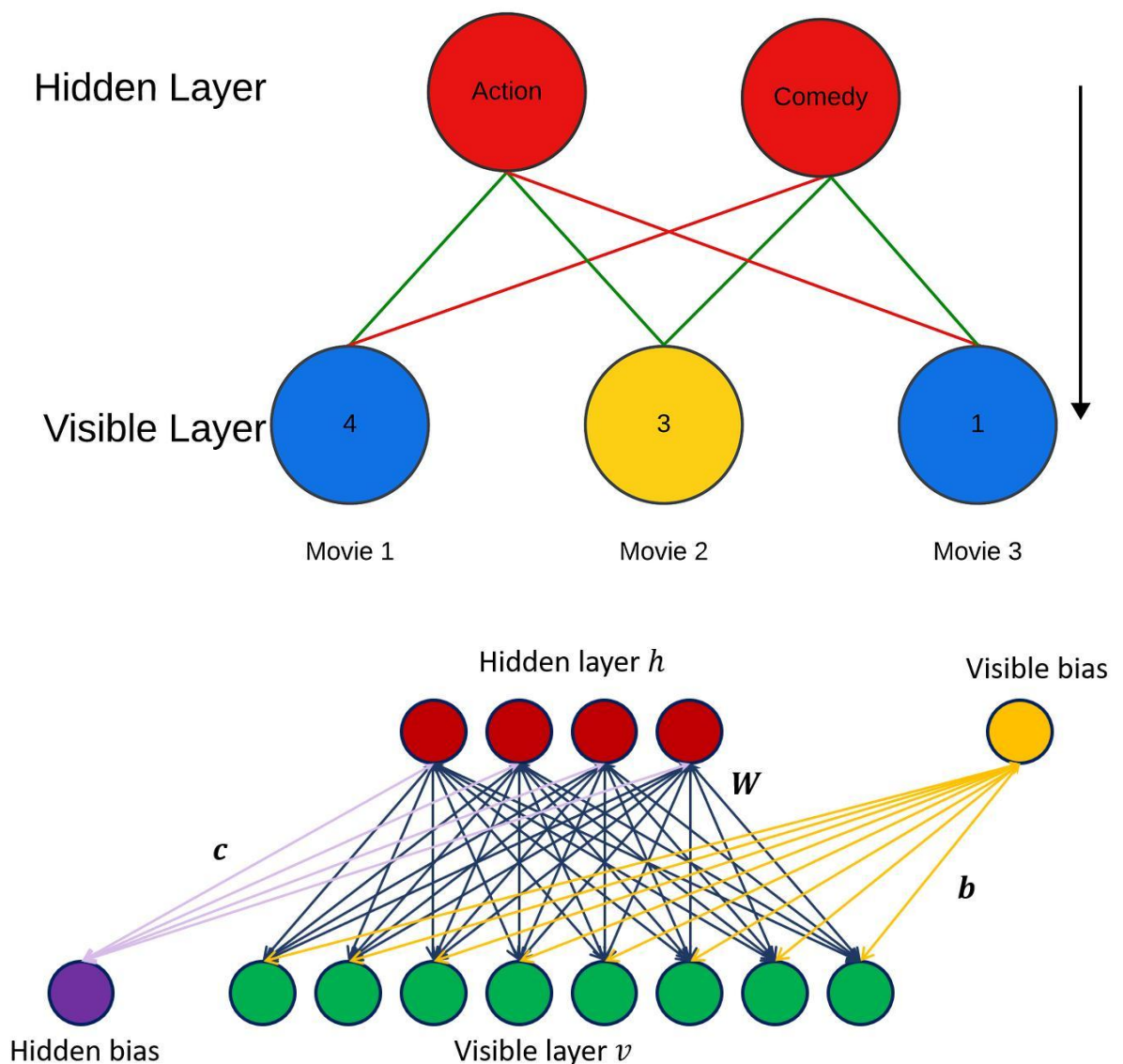
For facial images specifically, **Convolutional Neural Networks (CNNs)** are usually preferred because they learn spatial image features.

Conclusion

Improve the facial recognition system by using a **deeper CNN/neural network**, appropriate width, and nonlinear activations such as **ReLU**. The deeper architecture can learn increasingly complex facial representations.

4. Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks

A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.



4

Visible layer

The visible neurons represent movies or user interactions.

For example:

Movie	User interaction
Avengers	1
Titanic	0
Interstellar	1
Batman	1

Here, 1 can represent watched/liked and 0 can represent not watched/not liked, depending on how the system is designed.

Hidden layer

The hidden neurons learn **latent preferences**.

For example, hidden units may capture patterns related to action, romance, science fiction, comedy, etc., without requiring these categories to be explicitly labeled.

Workflow

First create a **user–movie interaction matrix**:

Then:

During training, the RBM learns weights representing relationships between movies and hidden preference factors. Traditionally, techniques such as **Contrastive Divergence** are used to train RBMs.

For a particular user, known viewing information is supplied to the visible layer. Hidden units infer the user's latent interests, and the network estimates preferences for unseen movies.

For example, if a user frequently watches:

Avengers + Batman + Iron Man

the RBM may learn a latent preference associated with superhero/action content and assign high scores to similar unseen movies.

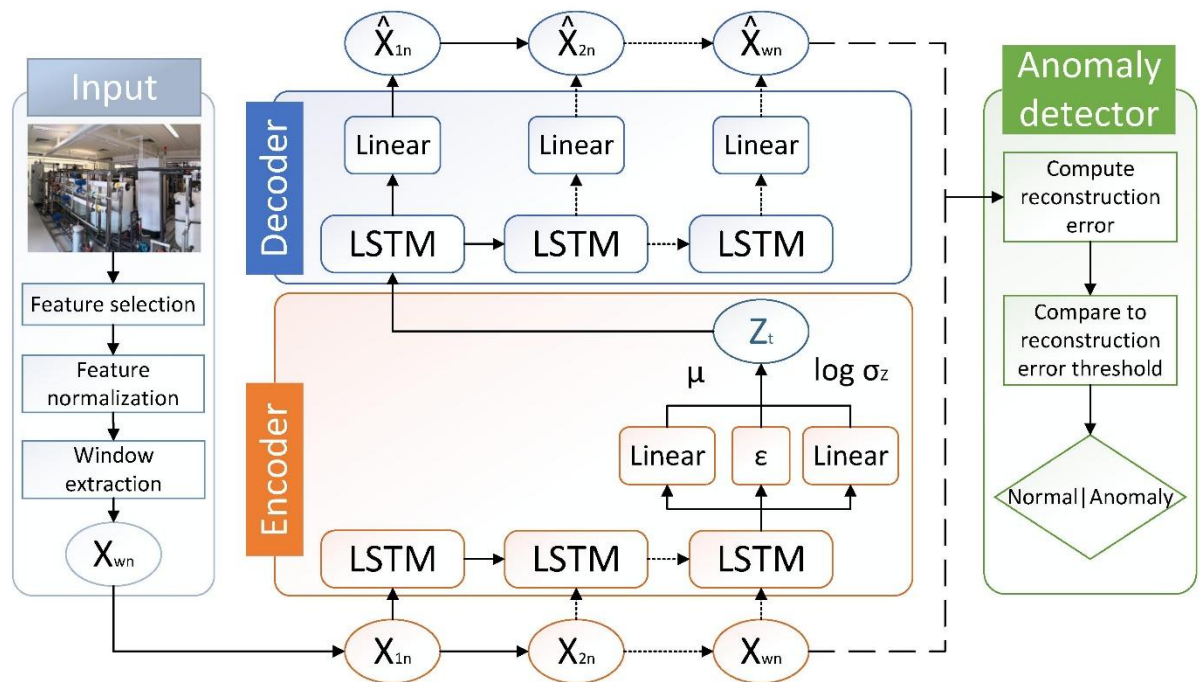
The movies with the highest predicted preference scores are recommended.

Conclusion

RBMs can build recommendation systems from user–movie interaction data without requiring explicit class labels because they learn **latent user preference patterns** through unsupervised learning.

5. Deep Learning Applications, Representation Learning

A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.



1. Data collection

Collect readings such as temperature, pressure, vibration, current, speed and other machine measurements every second.

Example:

2. Preprocessing

Clean the sensor data by handling missing values, reducing noise, normalizing values, and dividing continuous readings into suitable time windows.

3. Representation learning

Instead of manually defining every feature, a neural network can automatically learn useful representations.

For unlabeled data, an **autoencoder** is particularly useful:

The latent representation may capture relationships between temperature, vibration, pressure, etc.

4. Defect detection

There are two useful situations.

If labeled defective/normal examples are available:

The classifier can output:

If the probability exceeds a selected threshold, the product is flagged.

If labels are unavailable:

Train an autoencoder primarily on normal operating data.

Normal sensor pattern:

Abnormal pattern:

A large reconstruction error can therefore indicate a possible defect or abnormal machine state.

5. Real-time deployment

Because sensor readings arrive continuously:

When abnormal readings are detected, the system can flag the product, alert the operator, store the event for inspection, or trigger further quality checks.

Conclusion

A strong solution combines **representation learning** with neural networks. An autoencoder can automatically learn important sensor features and detect anomalies, while a supervised neural-network classifier can classify defects when labeled examples are available.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established

Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand